A Comparative Analysis of Innovation Diffusion Models Based on Real-World Data

Noam Cisneros

2026/27

Contents

1 Introduction **3**

2 Theoretical Basis **4**
2.1 Networks 4
2.2 Differential Equations 4
2.3 Game Theory - payoff matrices 5

3 The three Diffusion Models **5**
3.1 The Bass Diffusion Model 5
3.2 The Linear Threshold Model 8
3.3 Game-Theoretic Model 10
3.3.1 Update Rule 1: Coordination Games 10
3.3.2 Update Rule 2: Trends 11

4 Data Fitting **11**
4.1 The Dataset 11
4.2 The Optimisation Algorithm 12
4.3 The Bass Diffusion Model 12
4.4 The Linear Threshold Model 13
4.5 The Game-Theoretic Model 13

5 Results **14**
5.1 The Bass Diffusion Model 14
5.2 The Linear Threshold Model 14
5.3 The Game-Theoretic Model 15

6 Comparison **16**

7 Discussion **17**
7.1 Limits 17

8 Outlook **17**

A Appendix: symbols and naming conventions **18**

B Appendix: Python scripts **18**
B.1 Linear Threshold model 18

Todo list

write more about coordination games . . . 5
C in terms of y(0) . . . 6
not accurate . . . 7
fix this . . . 7
explain truncated normal distribution . . . 9
explain exactly what effect alpha has on the update rule . . . 11
check . . . 12
check . . . 12
WRONG: explain tol . . . 12
what is the interpretation of these optimal parameters? . . . 15
Summary, final fazit, etc. . . . 17
complete the table . . . 18
pseudocode for GTM . . . 19
insert URLs for the references . . . 20

1 Introduction

Innovation diffusion is a field of mathematical modeling that has been extensively studied in the past, and many models have been created to model these types of processes.

These different models describe the process of innovation diffusion on different levels of detail, with a greater or a lesser focus on the concrete decision process of the individual, and with different approaches to the general type of model used.

This work aims to analyze the advantages and disadvantages these different models have for different uses of innovation diffusion modeling. The models analyzed are mainly described in the work of Ye and Zeno on the topic [7], in which the most important models are laid out. This paper in large part inspired this work.

I am personally fascinated by the topic of innovation diffusion due to its huge variety of mathematical approaches, as well as its direct relation to real life. I additionally find it interesting to see how well the predictions of certain innovation diffusion Models are quite directly visible in everyday life; whenever a piece of information is spreading in a group of people, even when it only consists of a few individuals, one can quite easily observe all processes described by the mathematical model and how they actually influence decision process. In general, the persistence of mathematical patterns in human behavior is very interesting in my opinion, as they reveal how strongly collective human behavior follows mathematical patterns, which we oftentimes do not realize.

This strong occurrence of innovation diffusion theory is also the main reason for the relevance of this field of mathematics. Due to the theory of innovation diffusion being a quite generic theory, the applications of it range from fields like Sociology to economy, politics and many others.

In the following, I will first give a short theoretical introduction to some mathematical methods used to model innovation diffusion. Then, I will describe the three models of innovation diffusion that I will compare in this work, namely the Bass Diffusion model, a modified version of the Linear Threshold Model and a Game-Theoretic Model, and explain in detail how each of the models approaches the topic of innovation diffusion. Pseudocode for the implementation of the models as well as the GitHub repository with the Python code can be found in appendix B. After that, I will fit each of the models to a real dataset, and will compare how well they manage to describe the innovation process and how consistently they can be used. Finally, I will answer the following research question:

To what extent are the Bass Diffusion Model, the Linear Threshold Model, and the Game-Theoretic Model suitable for modelling real-world innovation diffusion, considering their computational efficiency, interpretability, and predictive capabilities?

2 Theoretical Basis

2.1 Networks

A common assumption in innovation diffusion models is that an individual's adoption decision depends on its social environment. Both the Linear Threshold Model and the Game-Theoretic Model incorporate this assumption through a network of interacting agents. Formally, such a network is represented by a graph consisting of a set of vertices (or nodes), $\mathscr{V}$, and a set of edges, $\mathscr{E}$, where each edge connects a start vertex to an end vertex. The network I am using is a so-called *directed graph*, meaning that it is not necessarily the case that if node A is connected to node B, node B is also connected to node A. For each node $i \in \mathscr{V}$, the set of neighbouring nodes is denoted by $\mathscr{N}_i$, which contains all nodes with which individual i can interact. The number of neighbours of node i is given by $d_i = |\mathscr{N}_i|$ and is referred to as the *degree* of node i in the network. This notation follows the convention used by Ye and Zino [7]. An example of how such a graph might look like can be seen in figure 1 Each node has a time-dependent state, denoted $\mathbf{x}_i(t)$. This value encodes wether or not the node has accepted the innovation or not, with $\mathbf{x}_i$ being equal to 1 when the innovation is adopted by agent i, and 0 otherwise. The fraction of agents that have adopted the innovation at time t is then equal to

$$|\mathscr{V}| \sum_{\rangle \in \mathscr{V}} \mathbf{x}_i(t)$$

and will be denoted by $\zeta(t)$.

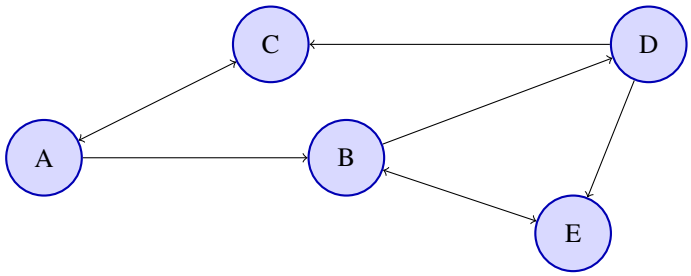


Figure 1: An example of a small directed network, with the following properties: $\mathscr{V} = \{A,B,C,D,E\}$, $\mathscr{E} = \{(A,C),(C,A),(A,B),(B,E),(E,B),(B,D),(D,E),(D,C)\}$

2.2 Differential Equations

When describing systems that change dynamically over time, Differential Equations (DEs) are oftentimes a helpful mathematical tool, as they can predict how a system will change in an infinitesimally small time-step, given only the state of the system in the current time-step. The Bass Diffusion Model is based on a Differential Equation (DE). A DE is an equation that relates a function to its own derivatives. When only the first derivative of the function and the function itself appears in the DE, it is called a *first-order Differential Equation*. The Bass Diffusion Model is such a first-order Differential Equation [2]. Since time is the only independent variable in the Bass Diffusion Model, the equation describing it is a so-called Ordinary Differential Equation (ODE).

DEs generally do not have solutions that can be expressed in closed form. If this is the case, their solutions can only be evaluated numerically with a given initial condition using algorithms such as the Runge-Kutta Method [4]. While the Bass Diffusion model does have a closed form solution (see section 3.1 for the derivation), I implemented it with a numerical algorithm for simplicity.

2.3 Game Theory - payoff matrices

An important concept in Game Theory and mathematical modeling in general are so-called payoff matrices. A payoff matrix encodes advantages and disadvantages of making certain decisions in a two-player coordination game with respect to the decision the opposing player takes. Such a payoff matrix can be written as follows:

$$\mathbf{P} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

where a is the reward for choosing to play 1 against an opponent who also chose 1, while c is the reward for playing 1 against an opponent who played 0. similarly, b is the reward for playing 0 against 1, and d is the reward for playing 0 against 0.

It should be noted that the matrix given above is not the most general notion of a two-player game that exists; rather, it encodes payoffs in a so-called *symmetric game*. a symmetric game is one where both players receive the same payoff for choosing the same actions against the same opponent action. a game that is not symmetric consists of two payoff matrices, one for each player, where the payoffs are not equal in general. Since such a model is unnecessarily complex for the purpose of modeling innovation diffusion, all games considered in this work will be assumed to be symmetric.

write more about co-ordination games

3 The three Diffusion Models

The three models I will compare in this work are the Bass Diffusion Model, which has a differential equation as its basis, The Linear Threshold model, which is based on a network of individuals with differently strong tendencies to adopt the innovation, and a version of a Game-Theoretic model, which is centered around the idea of two-player coordination games and following emerging trends.

3.1 The Bass Diffusion Model

The Bass Diffusion model is one of the first concrete mathematical models of innovation diffusion. It is defined by the ordinary differential equation

$$\frac{d\zeta}{dt} = (1-\zeta)(p+q\zeta) \tag{1}$$

where $\zeta(t)$ is the fraction of adopters at time t, and p and q are positive, real-valued parameters [2].

This differential equation can be solved via seperation of variables as follows:

$$\frac{d\zeta}{(1-\zeta)(p+q\zeta)} = dt.$$

Integrating both sides yields

$$\int \frac{d\zeta}{(1-\zeta)(p+q\zeta)} = t. \tag{2}$$

The Integral on the left-hand side of the equation can be rewritten by using partial fraction decomposition on the integrand:

$$\frac{1}{(1-\zeta)(p+q\zeta)} = \frac{A}{1-\zeta} + \frac{B}{p+q\zeta}$$
$$1 = A(p+q\zeta) + B(1-\zeta)$$
$$1 = Ap + Aq\zeta + B - B\zeta.$$

By comparing coefficients, one gets the two equations

$$Aq - B = 0$$

and

$$Ap + B = 1.$$

Solving this system of linear equations, A and B evaluate to

$$A = \frac{1}{p+q}$$
$$B = \frac{q}{p+q}.$$

Equation 2 can now be rewritten as follows:

$$\int \frac{1}{(p+q)(1-\zeta)} + \frac{q}{(p+q)(p+q\zeta)} = t.$$

factoring out the term $\frac{1}{p+q}$ and using the linearity of the integral to split it into the sum of two integrals gives

$$\frac{1}{p+q}\left(\int \frac{d\zeta}{1-\zeta} + q\int \frac{d\zeta}{p+q\zeta}\right) = t.$$

Both integrals can be solved using a simple u-substitution. The equation can then be solved for t:

$$\frac{1}{p+q}[\ln(p+q\zeta) - \ln(1-\zeta)] = t + C$$
$$\ln\left(\frac{p+q\zeta}{1-\zeta}\right) = (p+q)t + C$$
$$\frac{p+q\zeta}{1-\zeta} = e^{(p+q)t}\cdot C$$
$$p+q\zeta = e^{(p+q)t}\cdot C(1-\zeta)$$
$$p+q\zeta = e^{(p+q)t}C - e^{(p+q)t}C\zeta$$
$$p = e^{(p+q)t}C - e^{(p+q)t}C\zeta - q\zeta$$
$$p = e^{(p+q)t}C - \zeta\left(e^{(p+q)t}C + q\right)$$

The final expression for $\zeta(t)$ then becomes

$$\boxed{\zeta(t) = \frac{e^{(p+q)t}C - p}{e^{(p+q)t}C + q}}. \tag{3}$$

C in terms of y(0)

This function produces an S-shaped curve , with the parameters p and q determining how exactly the curve is shaped. Roughly speaking, p controls the steepness of the curve at its beginning, while q controls the steepness at the end . Figures 2 and 3 visualise the effect of the parameters on the diffusion curve.

not accurate

fix this

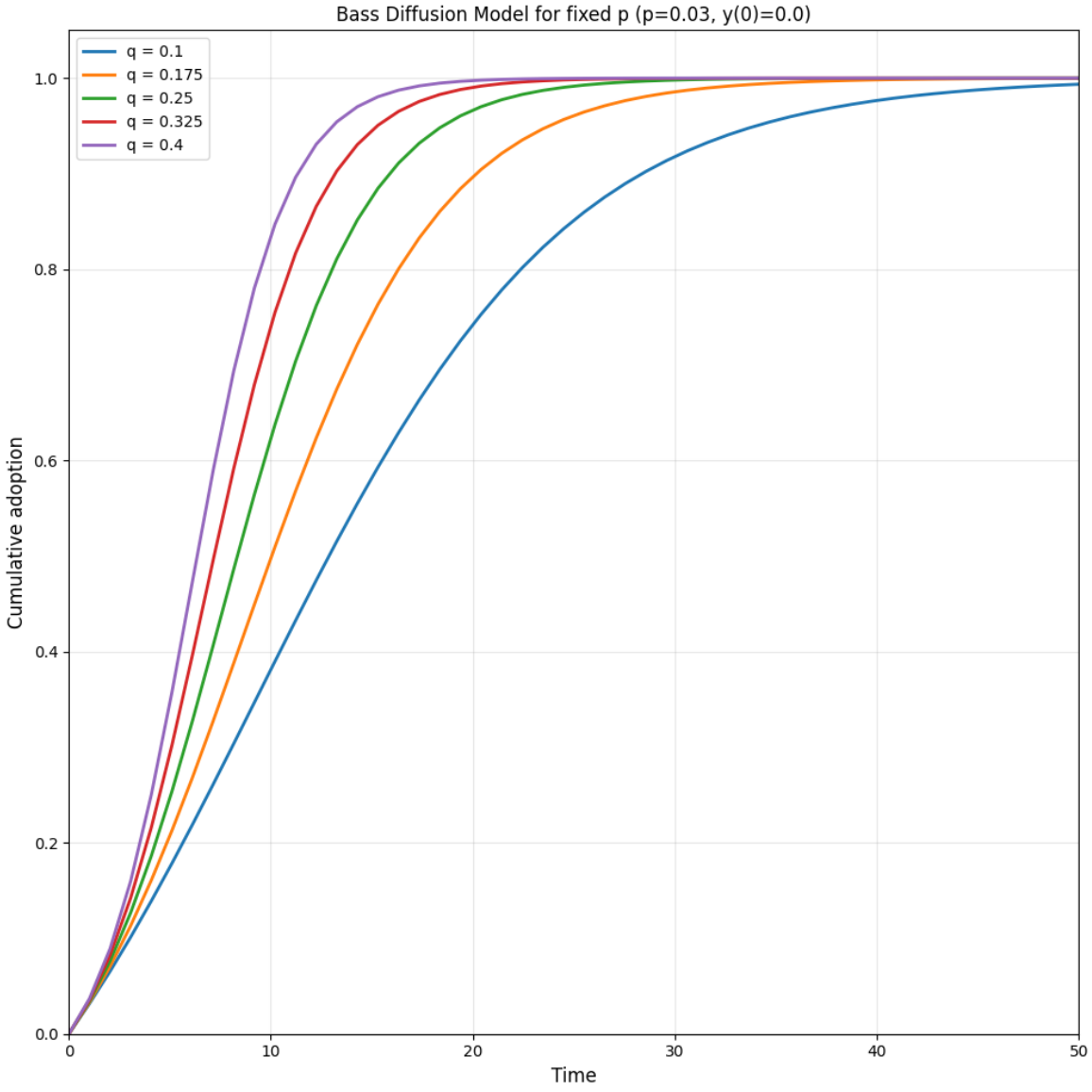


Figure 2: Bass Diffusion Model with fixed p.

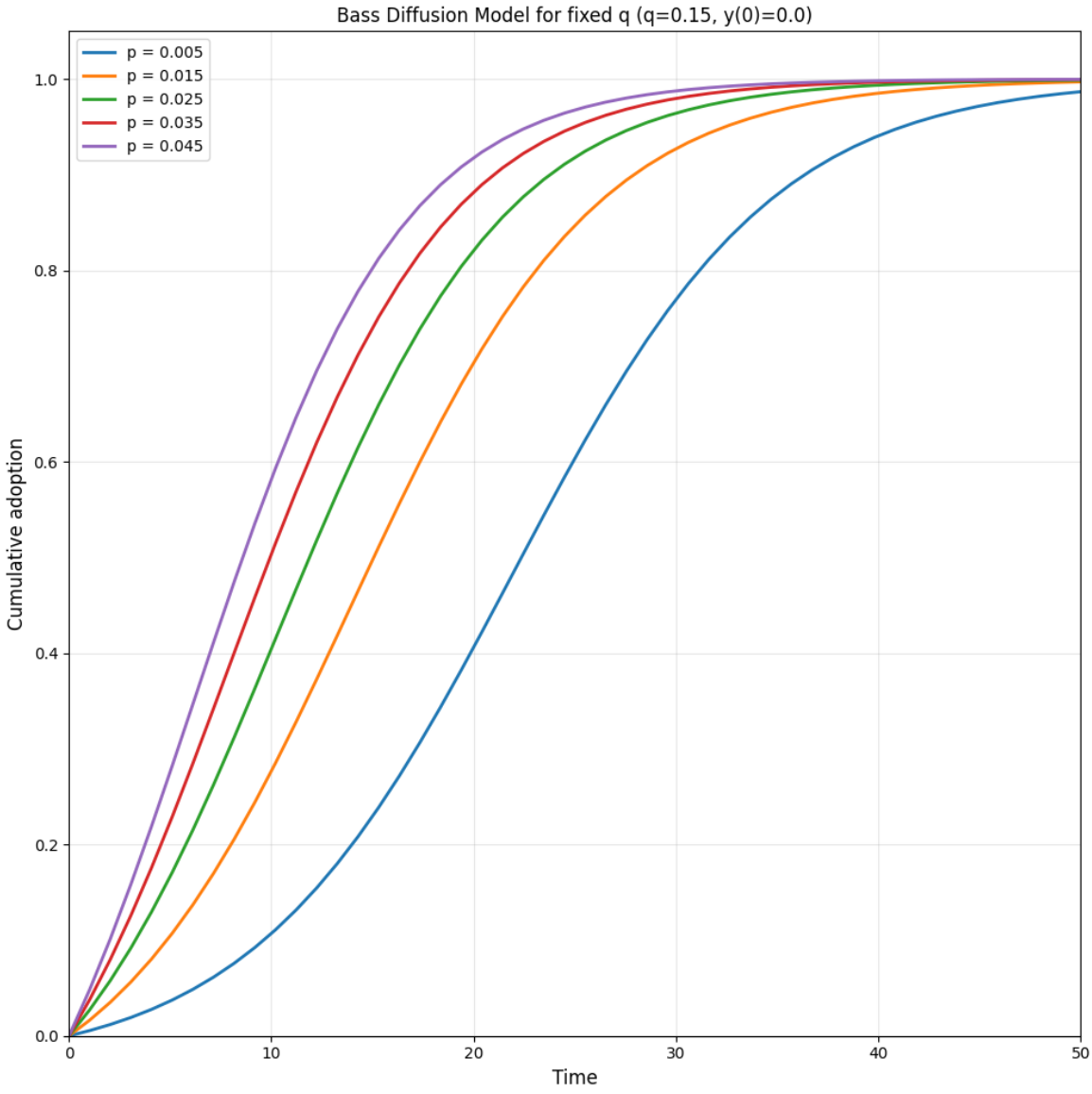


Figure 3: Bass Diffusion Model with fixed q.

3.2 The Linear Threshold Model

The Linear Threshold Model, henceforth referred to as LTM, initially proposed by Mark Granovetter in 1978 [3], is a diffusion model that is based on a network similar to the one described in section 2.1. Each node has a set of neighbours $\mathcal{N}_i$, and a threshold θ_i that lies in the interval $[0,1]$. The threshold describes how easily the individual accepts the innovation; in practice, it is the fraction of neighbours of the node that need to adopt the innovation in order for the node itself to adopt it as well [3]. Thus, the update rule for the LTM can be formalized as follows:

$$\mathbf{x}_i(t+1) = \begin{cases} 1, & \text{if} \quad \frac{1}{|\mathcal{N}_i|} \sum_{j \in \mathcal{N}_i} \mathbf{x}_j(t) \geqslant \theta_i \\ 0, & \text{if} \quad \frac{1}{|\mathcal{N}_i|} \sum_{j \in \mathcal{N}_i} \mathbf{x}_j(t) < \theta_i. \end{cases}$$

Recall from section 2.1 that $\mathbf{x}_i(t)$ stands for the state of node i at time t, where a state of 0 means that the agent has not adopted the innovation, while a state of 1 means that the agent has adopted the innovation. The sums in the update rule are equal to the average of the states of all neighbours of the node. When a node has accepted the innovation, it sets it threshold to 0, meaning that it cannot revert to the status quo.

This is the most basic Linear Threshold Model, described by Granovetter [3]. I did a few minor modifications to the model, in order to make it more practical for the specific purpose of fitting the curve to a dataset, as described in section 4.

The main way in which my Model differs from the one described by Granovetter [3]is that it does not update all nodes at every time-step. Instead, I defined a parameter $p \in [0;1]$ called the *update probability*, which is the probability that a node will update at a certain time-step. The reason for this is to mirror the fact that people do not realistically re-evaluate their decisions as often as the model assumes. It also makes the model's predictions more consistent and makes it easier and more intuitive to control how many time-steps the diffusion process should last.

In practice, it is not possible to assign the thresholds to each node manually, so they have to be sampled randomly. This is why in my model the thresholds are sampled from a normal distribution with a mean μ and a standard deviation σ . One problem with this is that a normal distribution allows outputs that are negative or greater than 1, while θ_i has to lie between 0 and 1. I fixed this issue by forcing the node to re-evaluate its threshold in the initialization phase of the simulation until a value between 0 and 1 is reached. This means that the final distribution used for the threshold evaluation is actually a so-called *truncated normal distribution*. One important consequence of this is that the parameters μ and σ are the mean and standard deviation of the original gaussian, *not* of the actual threshold distribution.

explain truncated normal distribution

The specific implementation I used works as follows:

first, the initial state of the population is determined by a set *seed fraction*; this parameter determines which fraction of the nodes adopt the innovation at time-step 0. All seed nodes will have a threshold of 0, meaning that it is impossible for them to revert to the status quo. Every node that is not marked as a seed node will start with the state 0, which means it will have the status quo.

After the set of seed nodes have been set, each node selects a set of neighbours by choosing a set of nodes randomly from the network. A node cannot chose itself as its own neighbour. Since it is possible that one node A has node B as its neighbour, while B does not have A as its neighbour, this produces a directed network as described in section 2.1. After the node has selected its set of neighbour nodes, it picks a threshold from the normal distribution as described above.
In total, the network has six parameters:

Symbol	Variable name in the code	meaning
d	`num_neighbours`	The number of neighbours of each node, or the degree of the network
$\lvert\mathscr{V}\rvert$	`num_agents`	The number of agents in the network
$\zeta(t=0)$	`seed_frac`	The fraction of all nodes acting as "seed nodes"
μ	`mean`	The mean value of the threshold distribution
σ	`stdev`	The standard deviation of the threshold distribution
p	`update_prob`	The probability that a node will update at any given time-step. This parameter is the same for every node.

Table 1: The parameters of the Linear threshold model

The most important parameters here are μ and σ, as they determine how easily the population adopts the innovation.

After the network has been initialized, the simulation process begins. In each step, the model loops

through each node in $\mathscr{V}$, and adjusts its state according to the update rule described at the beginning of the section. It is important to note that the node does not immediately change its state; if it were to do so, the order in which the nodes were updated would affect the outcome of the model. Instead, every updated state is stored in an array together with the index of the node it belongs to. After every node in the network has written its new state to the array, all nodes are actually updated using the data from the array. This leads to the nodes effectively updating at the same time.

For the details of the models implementation, see Appendix B.

It should be noted that this implementation of the LTM is quite a simple one; possible generalizations and improvements include assigning weight to each edge in the network based on how socially compatible two nodes are (a comparable idea has been investigated by Axelrod [1]), to introduce more than two possible states for the agents to chose from, and many more.

3.3 Game-Theoretic Model

Game Theory offers a good framework for innovation diffusion models, since its focus is on decision processes. When these processes are applied to individuals in a network that can decide between two states, 0 and 1, where 0 denotes adoption of the status quo and 1 denotes an adoption of the innovation, a diffusion process can be observed.

As proposed by Zino et al. [8], a time-varying network was used in the simulation. This means that each node re-evaluated its neighbours in each time-step of the simulation, which reduces the effect of the network structure on the curve of the diffusion process by approximating a randomly mixing population. Like for the LTM, I also used an update probability for this model.

The agents in the Game-Theoretic model can follow two update rules. Which one they follow is picked at random by each node at every time-step. The idea for this is taken from Zino et al. [8].

The first of the two update rules is based on a payoff matrix (see section 2.3, while the second update rule is based on following emerging trends. The probability that the second update rule is chosen by a node is called the *trend-sensitivity* of the network, and will be denoted s.

3.3.1 Update Rule 1: Coordination Games

When following the first update rule, the individuals will engage in a two-player coordination game with their neighbours. The node will decide whether or not to adopt the other node's state according to a payoff matrix.

Recall the definition of the generic payoff matrix introduced in section 2.3:

$$\mathbf{P} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

The total payoff for node i received for choosing action k (notated here as $\pi_i(x_i = k)$) is then defined simply as the sum of the payoffs received by the all of the neighbours of node i:

$$\begin{aligned} \pi_i(\mathbf{x}_i = 0) &= \sum_{j \in \mathscr{N}_i} x_j b + (1 - x_j) d \\ \pi_i(x_i = 1) &= \sum_{j \in \mathscr{N}_i} x_j a + (1 - x_j) c \end{aligned} \tag{4}$$

Assuming that the individuals always chose the option which returns the greatest payoff, the state of individual i at the next time-step will result from the following best-response update rule:

$$\mathbf{x}_i(t+1) = \arg\max_{k \in \{0;1\}} (\pi_i(x_i = k)).$$

This means that the Agent will always simply choose its next state such that the payoff π_i received for the choice is maximised.

The payoff matrix I used for my analysis was proposed by Zino et al. [8]. It simplifies the model by writing the payoff matrix as

$$\mathbf{P} = \begin{pmatrix} 1+\alpha & 0 \\ 0 & 1 \end{pmatrix},$$

Where the parameter α, called the *relative advantage* of the innovation, rewards the action of accepting the innovation. The reason for this is that the payoff for choosing to follow neighbours who adopted the innovation is increased relative to the payoff received for following neighbours who follow the status quo by the factor $1+\alpha$. This can be thought of as "increasing the influence" of the neighbours who accepted the innovation on the total payoff in equation 4.

explain exactly what effect alpha has on the update rule

3.3.2 Update Rule 2: Trends

Informally, he idea of this update rule is that an individual will adopt the innovation if it notices that the fraction of adopters has increased in the previous time-steps.

Concretely, individual i updates its state according to the following rule:

$$\mathbf{x}_i(t+1) = \begin{cases} 1 \text{ if } \zeta(t) - \zeta(t-1) > 0 \\ 0 \text{ if } \zeta(t) - \zeta(t-1) < 0 \\ \mathbf{x}_i(t) \text{ else} \end{cases}.$$

This update rule was also proposed by Zino et al. [8].

In total, the Game-Theoretic Model has the following parameters:

Symbol	Variable name in the code	meaning
d	`num_neighbours`	The number of neighbours of each node, or the degree of the network
$\lvert\mathcal{V}\rvert$	`num_agents`	The number of agents in the network
$\zeta(t=0)$	`seed_frac`	The fraction of all nodes acting as "seed nodes"
α	`alpha`	The relative advantage of the innovation. Increases the payoff of playing 1 against an opponent who also played 1.
s	`trend_sensitivity`	The probability that a node will update based on trends rather than payoff.
p	`update_prob`	The probability that a node will update at any given time-step. This parameter is the same for every node.

Table 2: The parameters of the Game-Theoretic model

4 Data Fitting

4.1 The Dataset

The dataset I used to fit the models to comes from a collection of datasets about the spread of new technologies in the US, supplied by "Our World in Data" [6]. The used the dataset about the fraction

of US households owning a refrigerator, labelled `Refrigerator` in the dataset collection. The dataset contains data from 1920 to 2017 , but I only used the data from 1925 to 1980 , since a plateau is already reached roughly around 1960. It should be noted that every node in the simulation of the LTM and Game-Theoretic Model will represent a single household, not an individual. There are several reasons for why I chose this specific dataset. First of all, the data shows a very clear adoption curve, with the adopter fraction starting at 0.01 in 1925 and reaching a plateau at 0.99. Another reason for why this dataset is well-suited for this analysis since the decision whether or not to buy a refrigerator is actually made by the households themselves in most cases, and social influences can be assumed to contribute to this decision.

check

check

4.2 The Optimisation Algorithm

Every one of the three models has a certain set of parameters, or inputs, that determine the prediction it makes. When fitting a specific model to the dataset, the idea is to find the optimal values for these three parameters such that the model's output aligns with the dataset as closely as possible. To find these optimal parameters, I used the `differential_evolution` optimisation algorithm from the Python package `scipy`, with a mean-squared error function.

What this means is that the model is initialized with random parameters, and given an error score, which rates how well the model predicted the dataset. This score is evaluated as follows:

$$E = \frac{1}{n}\sum_{i=0}^{n}(y_i - \hat{y}_i)^2,$$

where n is the length of the dataset, y_i is the models prediction for ζ at the ith time-step, and $\hat{y}_i$ is the actual fraction from the dataset. After this error is evaluated, the `differential_evolution` algorithm then "mutates" the parameters in a specific way in order to find parameters that produce a lower error, in a similar way to how genetic mutation leads to species adopting to their environment in nature [5].

The `differential_evolution` method takes the following parameters:

- `bounds`: the minimum and maximum values for each of the parameters that are being optimised.I determined the bounds by manually testing the model with different values and seeing in which range the parameters should roughly lie.
- `maxiter`: the maximum number of iterations the algorithm is allowed to run before having to return a result. This value was always set to 1000.
- `tol`: the *tolerance* of the algorithm. The smaller this number is, the closer the algorithm will try to get to the dataset. This value was always set to 0.001.

WRONG: explain tol

4.3 The Bass Diffusion Model

The Bass Diffusion model was the easiest model to optimise, since it only has two parameters and runs extremely quickly. The optimisation for the Bass model was run with the bounds

parameter	minimum	maximum
p	0.0001	1
q	0.0001	1

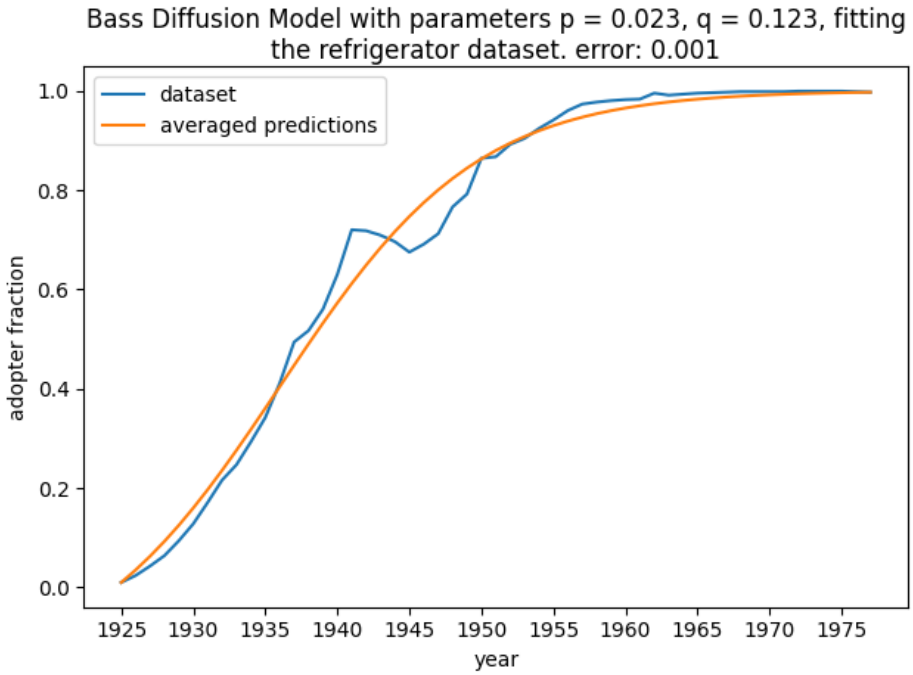


Figure 4: Output of the Bass Diffusion Model fitting the dataset.

4.4 The Linear Threshold Model

For the data fitting, the thresholds were again sampled from a normal distribution that is the same for all nodes, as described in Section 3.2. The number of Agents was fixed to 500, since this is the largest network which my computer could simulate reasonably fast. After manually experimenting with the parameter d, I found that a value of 10 produced good results. There were three parameters that I optimized, namely the standard deviation σ and the mean μ of the distribution, as well as the update probability p. I used the following ranges for the optimisation algorithm:

parameter	minimum	maximum
μ	0.001	0.8
σ	0.001	0.3
p	0.001	0.6

4.5 The Game-Theoretic Model

The Game-Theoretic model was considerably harder to optimise than the Linear Threshold Model, mainly because the relative advantage α can take any value from 0 to infinity. This means that I had to find the approximate upper bound manually through a trial-and-error process. I eventually arrived at the conclusion that an α value between 0 and 2 would produce a consistent result with the correct values for p and s. This led me to the following bounds:

parameter	minimum	maximum
α	0.0	2.0
s	0.1	0.3
p	0.001	0.4

5 Results

This section concerns itself with the general characteristics of each model. The models will be investigated mainly with respect to how easy or practical they are to use, which errors or complications they produce, and for which populations they produce a viable result.

5.1 The Bass Diffusion Model

The optimal parameters for the Bass Diffusion model are

p	q
0.023	0.123

Table 3: Optimal parameters for the Bass Diffusion Model

5.2 The Linear Threshold Model

Optimising the LTM to the dataset yielded a minimum of the MSE score with the following parameters:

p	μ	σ
0.226	0.085	0.054

Table 4: Optimal parameters for the LTM

The low value of μ means that the individuals are required to accept the innovation quite easily in order for the model to produce a curve that fits the dataset closely.

It should be noted that the average MSE error is the average of the errors for each run. The same is true for the Game-Theoretic Model in the next section.

One big issue I encountered when running the LTM with these parameters is that the results vary quite strongly. This was, however, fixed by introducing the update probability p described in section 3.2. Still, randomness is part of the model, so I ran the model 50 times with the same parameters and used the average of all the resulting curves to get the final value for the average error of the model. This process is visualised in figure 5, where the gray curves are the actual runs of the model, while the orange line is the average.

After fixing these issues, I managed to get the LTM to predict the real dataset quite closely, with an average Score of about 0.004. Figure 5 shows the final fit of the LTM.

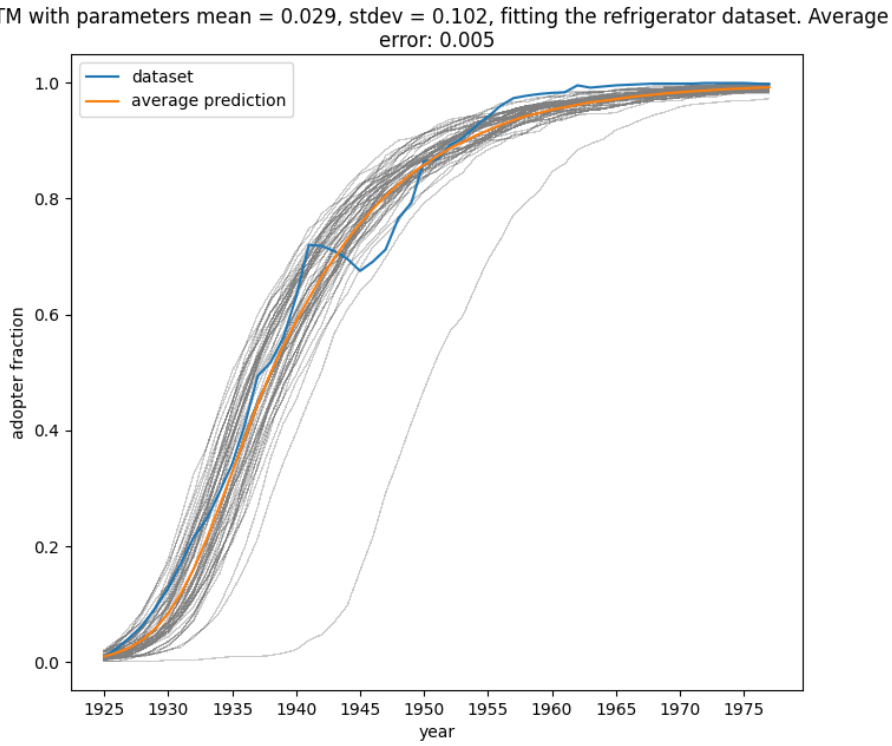


Figure 5: Averaged outputs of the LTM fitting the dataset.

5.3 The Game-Theoretic Model

After running the Game-Theoretic Model through the same optimisation algorithm, I found the following set of ideal parameters:

p	α	s
0.111	1.305	0.348

Table 5: Optimal parameters for the Game-Theoretic Model

what is the interpretation of these optimal parameters?

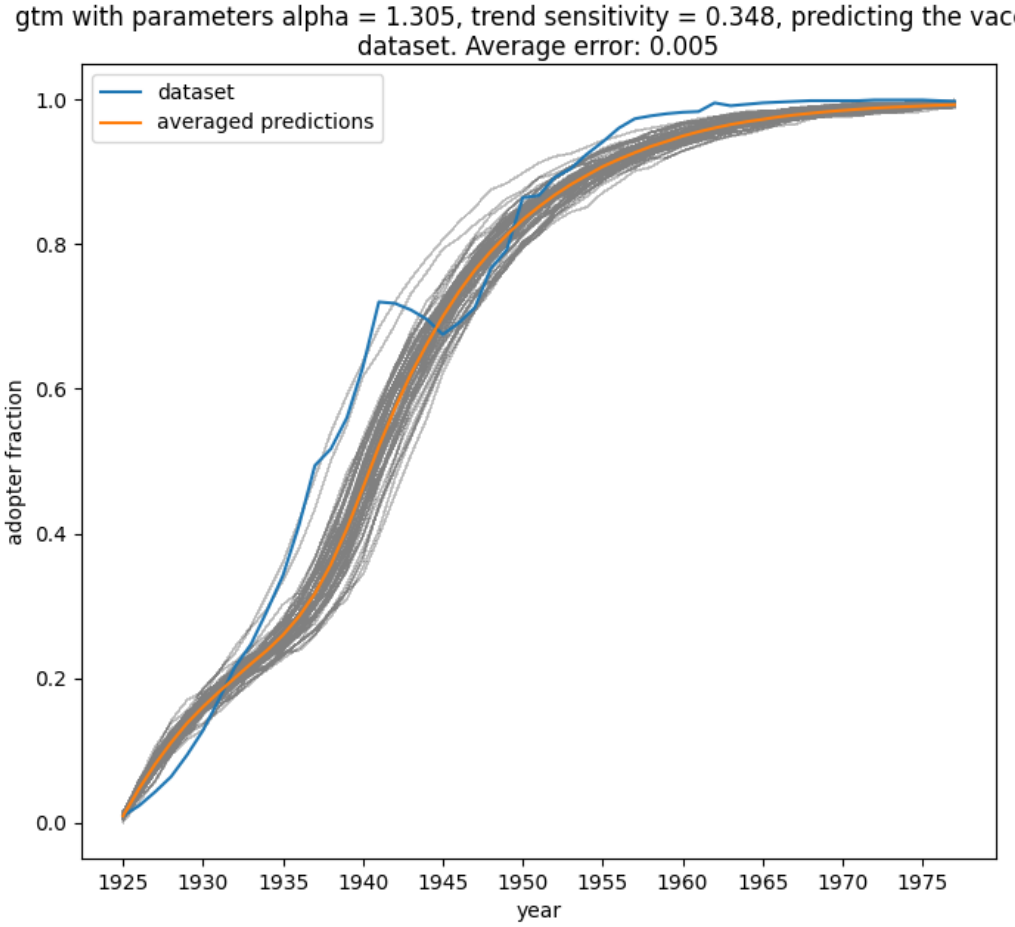


Figure 6: Averaged outputs of the Game-Theoretic Model predicting the `BCG Vaccine_Share of Population_PK` dataset.

6 Comparison

The three models will be compared with respect to the following aspects:

- *Fitting accuracy:* How well could each model fit the data?
- *Computational efficiency:* How efficient are the models computationally?
- *Realism:* How realistic are the assumption of each model?
- *Interpretability:* How well can the parameters of the models be interpreted?
- *Modelling capabilities:* Which adoption processes can the models describe?

7 Discussion

7.1 Limits

While the dataset used for data fitting in section 4 does model a general diffusion process, there are some factors contributing to the curve of the data that the models do not account for. The models assume that the decision to buy a refrigerator is only influenced by the social environment of the household in question, while factors such as the affordability of refrigerators are ignored.

8 Outlook

Summary, final fazit, etc.

A Appendix: symbols and naming conventions

The following table contains important symbols and their meanings.

Symbol	Meaning
$\zeta(t)$	The relative fraction of adopters at time t
$N(t)$	The absolute number of adopters at time t
p	The probability of a node to update its state at any given time-step
Linear Threshold Model	
$\mathscr{V}$	Set of nodes in the network
$\mathscr{E}$	Set of Edges in the network
$\mathscr{N}_i$	Set of neighbours of Node i
d_i	The degree of node i, or $\lvert\mathscr{N}_i\rvert$
Game-Theoretic Model	
$\mathbf{P}$	Payoff matrix
α	Relative advantage of the innovation

Table 6: naming conventions

complete the table

B Appendix: Python scripts

B.1 Linear Threshold model

The Pseudocode for the implementation of the LTM found on GitHub looks as follows:

Algorithm 1 The Linear Threshold Model

for i in $\mathscr{V}$ **do**
  $\theta_i \leftarrow$ random value sampled from $\mathscr{N}(\mu, \sigma)$ such that $0 < \theta_i < 1$
  $\mathbf{x}_i \leftarrow 1$ with probability p = seed fraction, 0 otherwise
  $\mathscr{N}_i \leftarrow$ select a random set of neighbours from $\mathscr{V}$
end for

while executed iterations $<$ maximum iterations **do**
  for i in $\mathscr{V}$ **do**
    if a random value is below p **then**
      evaluate $\mathbf{x}_i(t+1)$ based on the update rule
      if $(x)_i(t+1) = 1$ **then**
        Set θ_i to 0
      end if
      store the result in the state array
    else
      store the current state $\mathbf{x}_i(t)$ in the state array
    end if
  end for
  for i in $\mathscr{V}$ **do**
    update the state $\mathbf{x}_i$ using the corresponding value in the state array
  end for
end while

pseudocode for GTM

insert URLs for the references

References

[1] R. Axelrod. "The Dissemination of culture". In: *Journal of conflict resolution* 41.9 (1997), pp. 203–226.

[2] F. Bass. "A New Product growth for Model Consumer Durables". In: *Management Science* 15 (1969), pp. 215–227. URL: https://pdodds.w3.uvm.edu/files/papers/others/1969/bass1969a.pdf.

[3] M. Granovetter. "Threshold Models of Collective Behavior". In: *The American Journal of Sociology* 83.6 (1978), pp. 1420–1443.

[4] C. Runge. "Ueber Die Numerische Auflösung Von Differentialgleichungen". In: *Mathematische Annalen* 46.2 (1895), pp. 167–178.

[5] R. Storn and K. Price. "Differential Evolution – A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces". In: *Journal of Global Optimization* 11 (1997), pp. 341–359.

[6] Various sources. *Technology Diffusion (Comin and Hobijn (2004) and others)*. Processed by Our World in Data. Our World in Data, 2004.

[7] M. Ye and L. Zino. "Data-informed modeling of the formation, persistence, and evolution of social norms and conventions". In: *Handbook of Visual, Experimental and Computational Mathematics*. Ed. by B. Sriraman. Springer, 2023.

[8] L. Zino and M. Ye. "On Incentivizing Innovation Diffusion in a Network of Coordinating Agents". In: *IFAC-PapersOnLine* 56.2 (2023), pp. 4186–4191. URL: https://www.sciencedirect.com/science/article/pii/S2405896323021754.